

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 30 Minutes

Semester: Spring 2023
 Full Marks: 40

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. In the primary scholarship exam in Bangladesh, four lakh ($n=4,00,000$) students take part but only the top 50 students are given an award. 02
CO3

Write the asymptotic time complexity to give the awards. Assume that each award is given in a constant time.

- b. Write the asymptotic time complexity of the following function. 04
CO1

```

1. def contains_duplicates(elements):
2.     for outer in range(len(elements)):
3.         for inner in range(len(elements)):
4.             if outer == inner:
5.                 continue
6.
7.             if elements[outer] == elements[inner]:
8.                 return True
9.
10.    return False
  
```

- c. Express the following running time $T(n)$ with an asymptotic bound. 04
CO3

$$T(n) = 625T\left(\frac{n}{5}\right) + n^3$$

Any method is acceptable as long as you show calculations.

- 2 a. You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in ascending order, and after a specific position, they are in descending order. For example: [1, 3, 4, 5, 9, 6, 2, -1] 04
CO4

You have to find the maximum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?

- a) **Present** your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs. 04
 b) **Write** the time complexity of your algorithm. 01

b. In case of sorting an array in ascending order, *Bubble Sort* extends a sorted subarray at the rightmost side and, *Insertion Sort* extends a sorted subarray at the leftmost side. Your friend, Jimmy was told to find the first five largest and smallest numbers from a list of N distinct integers ($N > 10$). To solve the task, he modified the Bubble sort and Insertion sort algorithm for only 5 iterations and used the rightmost and leftmost 5 numbers as the 5 largest and smallest numbers respectively. Do you support his strategy? **Explain** with logical reasons. **02**

c. While sorting a list of 10 integers in ascending order using Quick Sort, after the first partition (using the first element as a pivot), the list looks like this : [13, 11, 19, 7, 23, 37, 29, 53, 59, 41]. Which element was the pivot before partitioning? **02**
Explain your answer in brief.

d. Consider the scenario in 2(c). **Show** how the list will look like after partitioning using **13** as the pivot. **01**

3 CO4 You are the coach of the renowned football team “Real Madrid”. Your team is behind one goal and there is still some time left to back in the game by scoring a goal. Benzema is your main striker and if you can pass the ball to him he will give the goal for sure. Currently the ball is in your goalkeeper’s (Courtois) hand. So, your main target should be Passing the ball in minimum steps from Courtois to Benzema as the time left is very low.

Courtois can pass the ball to Rudiger or Alaba
Rudiger can pass the ball to Modric
Alaba can pass the ball to Modric, Hazard or Nacho
Modric can pass the ball to Benzema
Hazard can pass the ball to Kroos
Nacho can pass the ball to Vini
Kroos can pass the ball to Benzema
Vini can pass the ball to Benzema

Now answer the following questions.

- a) Using a suitable algorithm, **find** the minimum number of passes your team will give to reach the ball from Courtois to Benzema. **Mention** the name of the algorithm, **Show** each step of the simulation properly to find out the minimum number of passes. **07**
- b) **Find** out the players who are required for this minimum number of passes scenario. **03**

Answer only one of 4, 4(or)

4 Suppose the company, "Sparkling Ideas," has been new in the market for the past few years and you work as a financial analyst hired for this company.

You have been provided a list of the company’s monthly incomes over the last year. However, the incomes are mixed with positive and negative figures, which indicates the company has faced some difficulties during certain months. The company's CFO has a special skill to identify the trend in the company's income details, which helps her make informed decisions. She believes that there is a particular set of consecutive months where the company earns the most, and she wants you to find out which one(s) it is.

Your task is to find the maximum earnings from the given monthly incomes and determine in which part of the year the company earned the most.

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
7	-15	9	-4	12	-8	3	-11	16	-2	6	-10

CO4

The figures are in millions of USD.

07

CO4

- Try to think of an efficient algorithm to solve this problem and provide the result in terms of the starting and ending months, along with the corresponding total income. **Show** a simulation of your algorithm.
- Calculate** the time complexity of your algorithm. Show proper mathematical logic.

03

4 or

Inspired by Karatsuba's algorithm for multiplying integers, your friend has come up with the following divide-and-conquer algorithm for squaring n -digit numbers. Pseudocode for the algorithm is given below. Your job is to fill in the details. You may assume that n is a power of two.

Algorithm 1: KARAT-SQUARE(x)	
Input : An n -digit number x	
Output : x^2	
1 if $n = 1$ then	
2 return x^2	
3 end	
4 $a \leftarrow$ the leftmost $n/2$ digits of x ;	
5 $b \leftarrow$ the rightmost $n/2$ digits of x ;	
6 $S_1 \leftarrow$ KARAT-SQUARE(a);	/* S_1 equals a^2 */
7 $S_2 \leftarrow$ KARAT-SQUARE(b);	/* S_2 equals b^2 */
8 $S_3 \leftarrow$ KARAT-SQUARE(_____);	/* fill in the blank */
9 $P \leftarrow$ _____;	/* P should equal $2ab$ */
10 return _____ $\times 10^n + P \times 10^{n/2} +$ _____;	

Now carefully read the algorithm above, and answer the following questions.

CO4

- Write x in terms of a and b . Looking at lines 4 and 5 of the algorithm should help.
- Square your answer from (i). The formula $(m + n)^2 = m^2 + 2mn + n^2$ should help.
- Fill in the blank in line 8. Your answer should involve a and b in some way. The idea is to compute a square that, along with a^2 and b^2 , helps you compute $2ab$. You may want to think about questions (iii) and (iv) at the same time.
- Fill in the blank in line 9. Your answer should involve S_1 , S_2 , and S_3 in such a way that P equals $2ab$.
- Fill in the blank in line 10.
- Write the running time of your algorithm above.
- Say you have another algorithm KARAT-SQUARE-2(x) that takes an n -digit number and squares it by recursively squaring six $n/3$ -digit numbers, and combining them in $O(n)$ time. Which algorithm is asymptotically faster: KARAT-SQUARE(x) or KARAT-SQUARE-2(x)?

0.5

0.5

02

03

01

01

02

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 30 Minutes

Semester: Spring 2023
 Full Marks: 40

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. In the primary scholarship exam in Bangladesh, two lakh ($n=2,00,000$) students take part but only the top **25** students are given an award. **02**

CO3

Write the asymptotic time complexity to give the awards. Assume that each award is given in a constant time.

- b. Write the asymptotic time complexity of the following function. **04**

CO1

```
1. def cumulative_sum(elem):
2.     for outer in range(len(elem)):
3.         for inner in range(outer+1, len(elem)):
4.             elem[outer] = elem[outer] + elem[inner]
5.
6.     return elem
```

- c. **Express** the following running time $T(n)$ with an asymptotic bound. **04**

CO3

$$T(n) = 25T\left(\frac{n}{5}\right) + n^3$$

Any method is acceptable as long as you show calculations.

- 2 a. You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in descending order, and after a specific position, they are in ascending order. For example: [9, 7, 6, 4, 2, 1, 3, 5, 8]

CO4

You have to find the minimum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?

- c) **Present** your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs. **04**
- d) **Write** the time complexity of your algorithm. **01**

b. In case of sorting an array in ascending order, *Bubble Sort* extends a sorted subarray at the rightmost side and, *Insertion Sort* extends a sorted subarray at the leftmost side. Your friend, Jimmy was told to find the first five largest and smallest numbers from a list of N distinct integers ($N > 10$). To solve the task, he modified the Bubble sort and Insertion sort algorithm for only 5 iterations and used the rightmost and leftmost 5 numbers as the 5 largest and smallest numbers respectively. Do you support his strategy?
CO2 **Explain** with logical reasons. **02**

c. While sorting a list of 10 integers in ascending order using Quick Sort, after the first partition (using the first element as a pivot), the list looks like this : [31, 19, 3, 17, 23, 37, 59, 61, 71, 43]. Which element was the pivot before partitioning?
CO2 **Explain** your answer in brief. **02**

d. Consider the scenario in 2(c). **Show** how the list will look like after partitioning using **31** as the pivot.
CO2 **01**

3 CO4 You are the coach of the renowned football team “FC Barcelona”. Your team is behind one goal and there is still some time left to back in the game by scoring a goal. Lewandowski is your main striker and if you can pass the ball to him he will give the goal for sure. Currently the ball is in your goalkeeper’s (Stegen) hand. So, your main target should be Passing the ball in minimum steps from Stegen to Lewandowski as the time left is very low.

Stegen can pass the ball to Alba or Roberto
 Alba can pass the ball to Torres
 Roberto can pass the ball to Busquets or Jong
 Busquets can pass the ball to Torres or Gavi
 Jong can pass the ball to Gavi
 Gavi can pass the ball to Lewandowski
 Torres can pass the ball to Lewandowski

Now answer the following questions.

- c)** Using a suitable algorithm, **find** the minimum number of passes your team will give to reach the ball from Stegen to Lewandowski. **Mention** the name of the algorithm, **Show** each step of the simulation properly to find out the minimum number of passes. **07**
- d)** Find out the players who are required for this minimum number of passes scenario. **03**

Answer only one of 4, 4(or)

4 Suppose the company, "Sparkling Ideas," has been new in the market for the past few years and you work as a financial analyst hired for this company.

You have been provided a list of the company’s monthly incomes over the last year. However, the incomes are mixed with positive and negative figures, which indicates the company has faced some difficulties during certain months. The company's CFO has a special skill to identify the trend in the company's income details, which helps her make informed decisions. She believes that there is a particular set of consecutive months where the company earns the most, and she wants you to find out which one(s) it is.

Your task is to find the maximum earnings from the given monthly incomes and determine in which part of the year the company earned the most.

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
17	-15	9	-4	12	-18	3	11	16	-2	-16	10

The figures are in millions of USD.

- CO4 c) Try to think of an efficient algorithm to solve this problem and provide the result in terms of the starting and ending months, along with the corresponding total income. **Show** a simulation of your algorithm. 07
- CO4 d) **Calculate** the time complexity of your algorithm. Show proper mathematical logic. 03

- 4 or Inspired by Karatsuba's algorithm for multiplying integers, your friend has come up with the following divide-and-conquer algorithm for squaring n -digit numbers. Pseudocode for the algorithm is given below. Your job is to fill in the details. You may assume that n is a power of two.

Algorithm 1: KARAT-SQUARE(x)	
Input : An n -digit number x	
Output : x^2	
1 if $n = 1$ then	
2 return x^2	
3 end	
4 $a \leftarrow$ the leftmost $n/2$ digits of x ;	
5 $b \leftarrow$ the rightmost $n/2$ digits of x ;	
6 $S_1 \leftarrow$ KARAT-SQUARE(a);	/* S_1 equals a^2 */
7 $S_2 \leftarrow$ KARAT-SQUARE(b);	/* S_2 equals b^2 */
8 $S_3 \leftarrow$ KARAT-SQUARE(_____);	/* fill in the blank */
9 $P \leftarrow$ _____;	/* P should equal $2ab$ */
10 return _____ $\times 10^n + P \times 10^{n/2} +$ _____;	

Now carefully read the algorithm above, and answer the following questions.

- viii) Write x in terms of a and b . Looking at lines 4 and 5 of the algorithm should help. 0.5
- ix) Square your answer from (i). The formula $(m + n)^2 = m^2 + 2mn + n^2$ should help. 0.5
- x) Fill in the blank in line 8. Your answer should involve a and b in some way. The idea is to compute a square that, along with a^2 and b^2 , helps you compute $2ab$. You may want to think about questions (iii) and (iv) at the same time. 02
- CO4 xi) Fill in the blank in line 9. Your answer should involve S_1 , S_2 , and S_3 in such a way that P equals $2ab$. 03
- xii) Fill in the blank in line 10. 01
- xiii) Write the running time of your algorithm above. 01
- xiv) Say you have another algorithm KARAT-SQUARE-2(x) that takes an n -digit number and squares it by recursively squaring six $n/3$ -digit numbers, and combining them in $O(n)$ time. Which algorithm is asymptotically faster: KARAT-SQUARE(x) or KARAT-SQUARE-2(x)? 02

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 20 Minutes

Semester: Spring 2024
 Full Marks: 35

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

1 a. Consider the following functions. 03

CO2

$$f_1(n) = \log(n!)$$

$$f_2(n) = n^2 \log_n(n^n)$$

$$f_3(n) = n^4 + 10n^2$$

$$f_4(n) = 17.13^n$$

$$f_5(n) = n \log n$$

$$f_6(n) = n * \sqrt[5]{n^2}$$

Write a correct asymptotic upper bound for each of the above and sort the functions in ascending order of their growth rate.

b. **Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning. 03

CO2

```
1. for i in range (1,n)
2.     j= 1
3.     while j < i*i
4.         j= j-1
```

c. **Express** the following (either one of the two) running time $T(n)$ with an asymptotic bound. 04

CO2

$$T(n) = 8T\left(\frac{n}{4}\right) + n\sqrt{n} \quad \text{Or} \quad T(n) = 2T(n-2) + 1$$

Any method is acceptable as long as you show calculations.

2 CO1 i. By showing necessary math, **explain** how Karatsuba's Fast Multiplication algorithm converts an 03

N-digit multiplication to three $N/2$ -digit multiplications.

ii. Can we modify the algorithm to multiply two N-bit binary numbers? **Explain** how or why not. 02

3 a. **Write** the worst case time complexity of quick sort? **Illustrate** an array where the worst case of 04

CO1

quick sort occurs if the last element is chosen as pivot.

- b. Consider an array containing N unique values where for some index i , the values are in increasing order from index 0 to $(i-1)$, and then again from i to $(N-1)$. Moreover, it is guaranteed that all the values from index 0 to $(i-1)$ are greater than all the values from i to $(N-1)$.

An example array is given below.

index	0	1	2	3	4	5	6	7
value	9	12	15	2	4	5	7	8

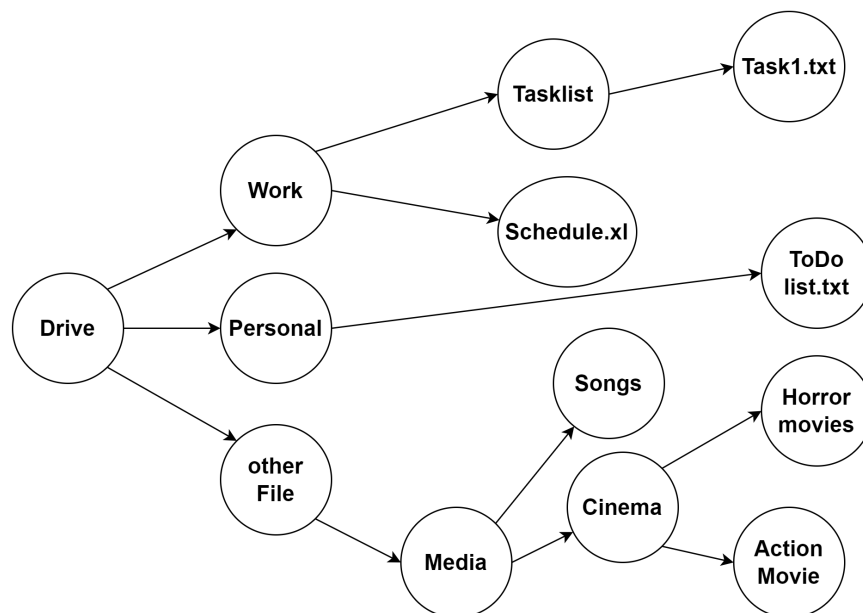
Here $i=3$, it means the values are in increasing order from index 0 to 2 , and then again from 3 to 7 . Also, all values from index 0 to 2 are greater than all values from 3 to 7 (it is guaranteed, no checking required).

Given such an array, propose an algorithm to find the index i .

- CO3 i) **Write** your algorithm with a code/pseudocode/flowchart/step-by-step instructions. 04
CO2 ii) **Write** the time complexity of your algorithm. 02

- 4 CO1 You are a computer science student and you are given a file structure encoded in a graph. You want to make a navigator system where if a person wants to find a file they can enter the name of the file and your program will return the exact location of the file.

For example, consider the following graph:



If the user wants to find task1.txt; your program will give the following output:
Drive->Work->Tasklist->task1.txt

If the user wants to find Horror Movies; your program will give the following output:
Drive->other file->Media->Cinema->Horror Movies

Now answer the following questions:

- Give** the adjacency list representation of this graph. You can use either the whole name or a shorter version of each node. 03
- Write** the name of your preferred algorithm to solve the above mentioned problem (a navigator system). Explain your reasoning in brief. 02
- Show** a simulation of your presented solution with proper use of data structure and other necessary details to give the desired answers as the sample input shows. 05

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 20 Minutes

Semester: Spring 2024
 Full Marks: 35

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. Consider the following functions. 03
 CO2

$$f_1(n) = \log(n!)$$

$$f_2(n) = n * \sqrt[4]{n^2}$$

$$f_3(n) = n^3 + 10n^2$$

$$f_4(n) = 17.13^n$$

$$f_5(n) = n \log n$$

$$f_6(n) = n^2 \log_n(n^n)$$

Write a correct asymptotic upper bound for each of the above and sort the functions in ascending order of their growth rate.

- b. Write the asymptotic time complexity of the following code snippet. Show your works/reasoning. 03
 CO2

```
1. for i in range (1,n)
2.     j= 1
3.     while j < i*i
4.         j= j+1
```

- c. Express the following (either one of the two) running time $T(n)$ with an asymptotic bound. 04
 CO2

$$T(n) = 4T\left(\frac{n}{4}\right) + \sqrt{n} \quad \text{Or} \quad T(n) = T(n-3) + n$$

Any method is acceptable as long as you show calculations.

- 2 CO1 i. By showing necessary math, **explain** how Karatsuba's Fast Multiplication algorithm converts an N-digit multiplication to three N/2-digit multiplications. 03
 ii. Can we modify the algorithm to multiply two N-digit hexadecimal numbers? **Explain** how or why not. 02

3 a. **Write** the worst case time complexity of quick sort? **Illustrate** an array where the worst case of quick sort occurs if the first element is chosen as pivot. **04**
CO1

b. Consider an array containing N unique values where for some index i , the values are in decreasing order from index 0 to $(i-1)$, and then again from i to $(N-1)$. Moreover, it is guaranteed that all the values from index 0 to $(i-1)$ are smaller than all the values from i to $(N-1)$.

An example array is given below.

index	0	1	2	3	4	5	6	7
value	5	4	1	12	10	9	7	6

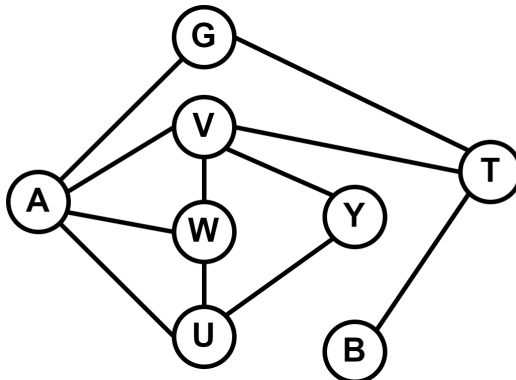
Here $i=3$, it means the values are in decreasing order from index 0 to 2 , and then again from 3 to 7 . Also, all values from index 0 to 2 are smaller than all values from 3 to 7 (it is guaranteed, no checking required).

Given such an array, propose an algorithm to find the index i .

CO3 i) **Write** your algorithm with a code/pseudocode/flowchart/step-by-step instructions. **04**

CO2 ii) **Write** the time complexity of your algorithm. **02**

4 CO1



- Give** the adjacency matrix representation of this graph. **03**
- Determine** whether the Graph is Bipartite/Bicolorable. Show a valid grouping/coloring of the vertices. **02**
- Show** a simulation of BFS algorithm with proper use of data structure and other necessary details to find the shortest path from A to B. **05**

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 30 Minutes

Semester: Spring 2025
 Full Marks: 30

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. Analyze the following code snippet and **express** the time complexity using the Big O-notation. You must mention the time complexity for each loop. **04**

```

1. int b = 0;
2. for (int i = n; i > 0; i -= 2) {
3.     for (int j = n; j > 0; j--) {
4.         b += i * j;
5.     }
6. }
7.
8. for (int k = 1; k < n; k *= 4) {
9.     for (int m = 0; m < n; m += 3) {
10.        if (m == 10) {
11.            break;
12.        }
13.        b += k + m;
14.    }
15. }
16. return b;

```

- b. You are given the following recurrence relation: **02**

CO2
$$T(n) = 3T\left(\frac{n}{4}\right) + \Theta(3n^2)$$

Solve the recurrence relation using an appropriate method

- c. **Prove** whether $5n^2 + 4n = \Theta(T(n))$, where $T(n)$ is the answer you found in part Q1(b). **03**

CO2

- d. Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? **Briefly** explain in your own words. **01**

CO2

2 Consider a farm with n cows of two colors, say, white cows and black cows. The cows are arranged in their barn in such a way that all white cows are kept first, followed by all black cows. Each of the cows is assigned a serial number. **You are to find the serial number of the first black cow.**

a. **Describe** how to convert the given information into an array of integers. 01
CO1

b. Given the array of integers from Q(2a), **present** a method with a time complexity **less than** 01
CO1 **$O(n)$** to solve the mentioned problem in the paragraph. Use pseudocode/ executable code/ step-by-step logical instructions to show your method.

c. Suppose we implement Quicksort using a strategy where we choose the **pivot as the** 03
CO1 **median of the first, last, and middle elements of a subarray**. To be specific, the pivot is selected as follows:

$$\text{pivot} = \text{median}(A[\text{st}], A[\text{end}], A[\text{mid}]) \quad \text{where} \quad \text{mid} = \left\lfloor \frac{\text{st} + \text{end}}{2} \right\rfloor$$

For example, consider the array $A = [6, 2, 9, 1, 7, 3, 8]$.

In the initial Quicksort call, $\text{start} = 0$ gives $A[0] = 6$, $\text{end} = 6$ gives $A[6] = 8$, and $\text{mid index} = \frac{(0+6)}{2} = 3$ gives $A[3] = 1$. Among these three values—1, 6, and 8—the sorted order is (1, 6, 8), so the median, and thus the chosen pivot, is 6.

What is the worst-case input for this pivot selection strategy in Quicksort? **Generate** such a worst-case scenario with a 7-element array containing the integers 1 through 7, each used exactly once. Briefly explain why it leads to worst-case behavior.

Now consider a factory where n humanoid robots work. You are to sort the robots based on some criteria. Answer Q2(d) – (e) based on this.

d. Each robot can form connections with some of the other robots. That means each robot can 02
CO1 form at most $(n-1)$ connections. Which algorithm will be the fastest to sort the robots based on the number of connections they formed? **Write** the name and time complexity of the algorithm.

e. Suppose, in the factory, there exists a system that can find the tallest robot in $O(1)$ time. 03
CO1 That means you are given a function, $\text{find_tallest}(A)$, which returns the tallest robot from a list called A , in constant time.

Considering this information, design an algorithm to sort the robots in descending order of their heights. **The time complexity of your algorithm must not exceed $O(n)$.**

- i. **Present** your algorithm using pseudocode/ executable code/ step-by-step logical instructions.
- ii. **Write** the time complexity of your algorithm.

3

In Rivendell, n voters are casting their votes for various political parties. The votes are recorded in an array, where each number represents a party's ID.

However, Rivendell follows a special rule: a party can only form the government if it receives more than $\lfloor \frac{n}{2} \rfloor$, that is, a strict majority. Otherwise, the election results in a deadlock, and a re-election must be held. For example, in the array $[1, 2, 2, 2, 3, 2, 1]$, party 2 wins the majority since it got 4 votes (more than $\lfloor \frac{7}{2} \rfloor = 3$).

- a. Can two different parties each secure a majority in any given vote array? **Explain** why or why not. **02**
- CO1
- b. Given a vote array and a party ID, we need to determine whether the party has won the majority in the array. **Propose** an $O(N)$ algorithm to verify this, where N is the number of voters. **02**
- CO1
- c. Now, we want to apply a divide and conquer approach to find the majority from a given vote array. Like merge sort, if we divide the vote array into two halves, can the majority element in each half be different from the majority of the entire array? **Justify** your answer with an example. **02**
- CO1
- d. Reuse your algorithm of Q3(b) to complete the following algorithm to find the majority party from the vote array. The algorithm should return -1 if there's no majority party. You can use either pseudo-code or step-by-step instructions. **04**
- CO1

```
function majority(arr):
    if length(arr) == 1:
        return arr[0]

    mid = length(arr) // 2
    left_majority = majority(arr[0 ... mid])
    right_majority = majority(arr[mid+1 ... length(arr)-1])

    # Merge Step: Reuse your algorithm of Q3(b) to complete
```

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 30 Minutes

Semester: Spring 2025
 Full Marks: 30

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. Analyze the following code snippet and **express** the time complexity using the Big O-notation. You must mention the time complexity for each loop. **04**
CO2

```

1. int b = 0;
2. for (int i = n; i > 0; i -= 3) {
3.     for (int j = 0; j < n; j += 3) {
4.         if (j == 50) {
5.             break;
6.         }
7.         b += i + j;
8.     }
9. }
10.
11. for (int k = 1; k < n; k *= 3) {
12.     for (int m = n; m > n; m /= 5) {
13.         b += k + m;
14.     }
15. }
16. return b;

```

- b. You are given the following recurrence relation: **02**
CO2

$$T(n) = 5T\left(\frac{n}{6}\right) + \Theta(3n)$$

Solve the recurrence relation using an appropriate method

- c. **Prove** whether $5n + 4 = \Theta(T(n))$, where $T(n)$ is the answer you found in part Q1(b). **03**
CO2

- d. Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? **Briefly** explain in your own words. **01**
CO2

2 Consider a farm with n sheep of two colors, say, white sheep and black sheep. The sheep are arranged in their barn in such a way that all white sheep are kept first, followed by all black sheep. Each of the sheep is assigned a serial number. **You are to find the serial number of the first black sheep.**

a. **Describe** how to convert the given information into an array of integers. 01
CO1

b. Given the array of integers from Q(2a), **present** a method with a time complexity **less than** 01
CO1 **$O(n)$** to solve the mentioned problem in the paragraph. Use pseudocode/ executable code/ step-by-step logical instructions to show your method.

c. Suppose we implement Quicksort using a strategy where we choose the **pivot as the** 03
CO1 **median of the first, last, and middle elements of a subarray**. To be specific, the pivot is selected as follows:

$$\text{pivot} = \text{median}(A[\text{st}], A[\text{end}], A[\text{mid}]) \quad \text{where} \quad \text{mid} = \left\lfloor \frac{\text{st} + \text{end}}{2} \right\rfloor$$

For example, consider the array $A = [6, 2, 9, 1, 7, 3, 8]$.

In the initial Quicksort call, $\text{start} = 0$ gives $A[0] = 6$, $\text{end} = 6$ gives $A[6] = 8$, and $\text{mid index} = \frac{(0+6)}{2} = 3$ gives $A[3] = 1$. Among these three values—1, 6, and 8—the sorted order is (1, 6, 8), so the median, and thus the chosen pivot, is 6.

What is the worst-case input for this pivot selection strategy in Quicksort? **Generate** such a worst-case scenario with a 7-element array containing the integers 1 through 7, each used exactly once. Briefly explain why it leads to worst-case behavior.

Now consider a factory where n humanoid robots work. You are to sort the robots based on some criteria. Answer Q2(d) – (e) based on this.

d. Each robot can form connections with some of the other robots. That means each robot can 02
CO1 form at most $(n-1)$ connections. Which algorithm will be the fastest to sort the robots based on the number of connections they formed? **Write** the name and time complexity of the algorithm.

e. Suppose, in the factory, there exists a system that can find the tallest robot in $O(1)$ time. 03
CO1 That means you are given a function, $\text{find_tallest}(A)$, which returns the tallest robot from a list called A , in constant time.

Considering this information, design an algorithm to sort the robots in descending order of their heights. **The time complexity of your algorithm must not exceed $O(n)$.**

- i. **Present** your algorithm using pseudocode/ executable code/ step-by-step logical instructions.
- ii. **Write** the time complexity of your algorithm.

3

In Transylvania, n audiences are casting their votes for various movies. The votes are recorded in an array, where each number represents a movie's ID.

The theater has a rule: for a movie to be selected for screening, it must receive more than $\lfloor \frac{n}{2} \rfloor$, that is, a strict majority. Otherwise, the theatre rejects this voting, and a re-election must be held. For example, in the array $[1, 2, 2, 2, 3, 2, 1]$, movie 2 wins the majority since it got 4 votes (more than $\lfloor \frac{7}{2} \rfloor = 3$).

- a. Can two different movies each secure a majority in any given vote array? **Explain** why or why not. **02**
- CO1
- b. Given a vote array and a movie ID, we need to determine whether the movie has won the majority in the array. **Propose** an $O(N)$ algorithm to verify this, where N is the number of audiences. **02**
- CO1
- c. Now, we want to apply a divide and conquer approach to find the majority from a given vote array. Like merge sort, if we divide the vote array into two halves, can the majority element in each half be different from the majority of the entire array? **Justify** your answer with an example. **02**
- CO1
- d. Reuse your algorithm of Q3(b) to complete the following algorithm to find the majority movie from the vote array. The algorithm should return -1 if there's no majority movie. You can use either pseudo-code or step-by-step instructions. **04**
- CO1

```
function majority(arr):  
    if length(arr) == 1:  
        return arr[0]  
  
    mid = length(arr) // 2  
    left_majority = majority(arr[0 ... mid])  
    right_majority = majority(arr[mid+1 ... length(arr)-1])  
  
    # Merge Step: Reuse your algorithm of Q3(b) to complete
```

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 30 Minutes

Semester: Spring 2026
 Full Marks: 35

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a.** Given two growth functions $G_A(n) = 30n + 8$ and $G_B(n) = n^2 + 1$ where $G_A(n) = O(n)$ and $G_B(n) = O(n^2)$. It is observed that (as shown in the following Table 1) for some small values of n , $G_A(n) > G_B(n)$. **02**
- CO2**

n	$G_A(n) = 30n + 8$	$G_B(n) = n^2 + 1$	Observation
3	98	10	$G_A(n) > G_B(n)$
4	128	17	
5	158	26	

Table 1

Given this scenario, **explain** why an algorithm A with growth function $G_A(n)$ is still considered more efficient than algorithm B with growth function $G_B(n)$.

- b.** **Determine** the time complexity of the **foo()** function in Big-O notation. **03**
- CO2**

<pre> int foo (int n) { int sum = 0; for(int i=1; i<=n; i*=4) { for(int j=1; j<=n; j+=7) { sum = sum + (i*j); for(int k=1; k<=n; k+=3) { if (k%2==0) break; } } } return sum; } </pre>	<pre> int recur(A[], p, r) { int n = (r-p+1); // size of the array if (n<=1) return 2; int b = recur (A[], p, p+($\frac{n}{4}$)); int c = recur (A[], p+($\frac{n}{4}$)+1, p+($\frac{n}{2}$)); int d = recur (A[], p, p+($\frac{n}{3}$)); int e = foo(n); return b+c-d+e; } </pre>
---	---

- c. In the **recur()** function of Q1(b), parameter **p** denotes the start index and parameter **r** denotes the end index of the array **A[]**. 02
+
03

Write the recurrence relation for the **recur()** function and **solve** it using any appropriate method to determine the asymptotic time complexity of the **recur()** function.

2

The Department of CSE at BRAC University maintains two student ID lists each semester to track students attending the final exam. **List A** holds the IDs of students registered for the regular final exam and **is always sorted**. **List B** holds the IDs of students registered for the makeup final exam and **is not always sorted**. The two lists for the current semester are given below.

List A:

2	3	7	11	13
---	---	---	----	----

List B:

5	1	3	3	4
---	---	---	---	---

You, as an algorithm enthusiast, decided to help the department by maintaining and arranging the total registered students for the final exam.

- a. First, arrange List B in ascending order **using a linear time complexity sorting algorithm**. 03
CO1 Show the **simulation** of the sorting algorithm on List B using all necessary steps.
- b. Now that both lists are sorted, **merge** them into one sorted **List C using linear time complexity** to track all registered students. **Show** the full simulation of the merging process using all necessary steps. Use your sorted B from part (a). 03
CO1
- c. After merging the lists into List C, you accidentally **rotated the list by 3 (three) positions to the left**, producing **List D**, which is no longer sorted (*You may draw List D for your convenience*). 03
CO1

You now want to sort List D using quicksort. To avoid a completely unbalanced first partition, you do not want to choose any element as pivot that would place all other elements on only one side after the first partition step (that is, one resulting subarray would be empty).

Design a **Binary Search** algorithm that finds, in **$O(\log_2 n)$** time, the **index or indices of the element(s) that should be avoided** as the first pivot choice from such a rotated list. **Write** your answer in pseudocode or programmable code.

- d. The department says that next semester, **makeup exam student IDs could be any integer up to 1000000000** due to a change in ID policy. However, the total number of students will remain the same as this semester. What **issue(s)** will arise with the algorithm used in Q2(a)? **Explain** in brief. 01
CO2

We are given an array A consisting of N positive integers. We want to find the Max-GCD Subarray - a contiguous subarray whose GCD (Greatest Common Divisor) of all elements is maximized.

Recall that,

- The GCD of two integers a and b , denoted $\text{GCD}(a, b)$ is the largest integer that divides both.
- The GCD of a single-element subarray is the element itself.
- Also, $\text{GCD}(0, a) = a$ for any integer a .

For example, $A = \{12, 18\}$ has three possible subarrays:

- $\{12\} \rightarrow$ single element, $\text{GCD} = 12$
- $\{18\} \rightarrow$ single element, $\text{GCD} = 18$
- $\{12, 18\} \rightarrow \text{GCD} = 6$

The Max-GCD subarray here is simply $\{18\}$ with $\text{GCD} = 18$,

Now, we have designed a divide-and-conquer algorithm for this problem, following the same structural concept as the Max-Sum Subarray algorithm, but modified for GCD:

```
function MaxGCDSubarray(A, left, right):
    if (left == right):
        return A[left]
    mid = (left + right) / 2
    LGS = MaxGCDSubarray(A, left, mid)
    RGS = MaxGCDSubarray(A, mid + 1, right)
    CGS = CrossGCD(A, left, mid, right)
    return max(LGS, RGS, CGS)

function CrossGCD(A, left, mid, right):
    leftGCD = 0
    g = 0
    for (i from mid down to left):
        g = GCD(g, A[i])
        if (g > leftGCD):
            leftGCD = g
    rightGCD = 0
    g = 0
    for (i from mid+1 to right):
        g = GCD(g, A[i])
        if (g > rightGCD):
            rightGCD = g

    return GCD(leftGCD, rightGCD)
```

- a.** For the input array $A = \{6, 10, 35, 15\}$, demonstrate where the above algorithm will work properly or not for the above problem. **Explain** your reasoning. **03**
- CO2**

- b.** We now want to apply an optimization: instead of computing $\text{GCD}(g, A[i])$ using the standard Euclidean algorithm (which runs in $O(\log N)$ time, where N is the size of the array A and $1 \leq A[i] \leq N$), we now call another built-in function $\text{Fast_GCD}(g, A[i])$, which returns the GCD answer in $O(\sqrt{N})$ time. You can directly assume it without finding any.

CO2

02

Write the recurrence relation to express the time complexity of the **MaxGCDSubarray** algorithm after applying the **Fast_GCD** function.

4

A space agency maintains a communication network among planetary relay stations. Each station can directly send signals to a set of nearby stations. Because of a solar storm, several relay constraints have been introduced, and the agency needs to analyze how the network behaves under these conditions. The network is modeled as a graph in Figure 1, where the relay stations are considered nodes, and the communication links are edges.

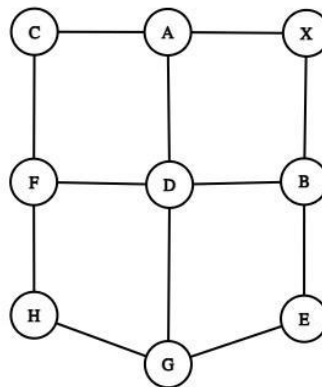


Figure 1

- a.** When a signal is transmitted from station X, it spreads outward such that in each time unit, all stations that just received the signal transmit it further.

CO1

04

Find out the minimum number of time steps required to transmit the signal from station X to station H. For tie-breaking on multiple eligible stations at the same time step, process them in alphabetical order. **Show** the simulation using the necessary data structures.

- b.** To avoid interference, the agency must assign either of two different frequencies (say, $freq_A$ and $freq_B$) to each station such that no two directly connected stations use the same frequency. Determine whether such an assignment is possible. **Present** your idea using a full simulation.

CO3

03

- c.** After the solar storm, **some** relay stations are **locked** for security. You cannot initially pass through a locked station unless you have already collected its required unlocking key from another rescuer station. Visiting a rescuer station gives you the key to one specific locked station.

CO3

03

Suppose you are given lists for the locked and the rescuer stations. At the beginning, you have no keys except for station X. It is guaranteed that the unlocking steps don't lead to an infinite loop, and a locked station doesn't provide any unlocking key.

Explain how the algorithm from Q4(a) should be modified so that it can still find the minimum number of time steps needed to send the signal from station X to station H.